

■ Telegram Mini App

Complete Deployment Guide

From Zero to Live — A to Z

■ What's Inside

1. Buying VPS Hosting
2. Installing AlmaLinux 10 Server
3. Installing Node.js, MySQL, Nginx
4. Connecting Your Domain
5. Setting Up SSL (Free HTTPS)
6. Importing Database
7. Configuring Code (Replacing Placeholders)
8. Creating Telegram Bot
9. Going Live with PM2
10. Testing & Troubleshooting

Tested on: AlmaLinux 10 (also works on RHEL 9 / Rocky Linux 9 / CentOS Stream 9)

Difficulty: Beginner-friendly — no prior server experience needed

Time: ~2 hours from start to live

Created by Mr Naveed Shah

YouTube: @shahgideas (SHAHG)

■ Table of Contents

#	Section	Page
1	Prerequisites & What You'll Need	3
2	Buying a VPS — Recommended Providers	4
3	Initial Server Setup (AlmaLinux 10)	6
4	Installing Node.js, MySQL, Nginx	8
5	Connecting Your Domain to VPS	11
6	Setting Up Free SSL with Certbot	13
7	Setting Up Database (Import & Configure)	15
8	Configuring Project Code	17
9	Creating Your Telegram Bot	19
10	Configuring Nginx (Reverse Proxy)	21
11	Deploying with PM2	24
12	Linking Bot to Mini App	26
13	Going Live — Final Tests	27
14	Troubleshooting Common Issues	29
15	Maintenance & Updates	32
★	About the Author — Mr Naveed Shah	34

1. Prerequisites & What You'll Need

Before starting, make sure you have the following ready. This guide assumes you have **no prior server experience** and walks you through every single step.

✓ Required Items

Item	Why You Need It	Cost
A computer with internet	To run commands and configure the server	—
A credit/debit card	To buy VPS and domain	—
Email address	For VPS account, domain, Telegram bot	Free
Telegram account	To create the bot via @BotFather	Free
VPS Server	To host the app (we'll buy this in Section 2)	\$4–10/month
Domain name	Like 'yourapp.com' (we'll buy this too)	\$1–15/year
~2 hours of time	To complete the full setup	—

■ **NOTE:** If you don't have a domain yet, you can buy one from **Hostinger, Namecheap, or GoDaddy**. We recommend Hostinger for easy DNS management. You can use any TLD like .com, .net, .xyz, .live, etc.

✓ Skills (Don't Worry — All Beginner Level!)

You only need to know how to:

- Copy and paste commands into a terminal
- Click links and fill out web forms
- Read instructions carefully and follow them step-by-step

■ **TIP:** Each command in this guide is provided exactly as you need to type it. You won't write a single line of code from scratch — just copy, paste, and adjust your specific values.

2. Buying a VPS — Recommended Providers

A VPS (Virtual Private Server) is your own computer in the cloud where the app will run 24/7. Below are tested providers ranked by quality and value:

Provider	Min Plan	Why Choose
Hostinger VPS	\$4.99/mo	Best for beginners, includes free domain & easy setup
Contabo	\$5.99/mo	Cheapest with most resources (great for high traffic)
DigitalOcean	\$6/mo	Most reliable, used by professionals worldwide
Vultr	\$6/mo	Many global locations, fast deployment
Linode (Akamai)	\$5/mo	Excellent performance, great support

✓ Recommended Specifications

For a Telegram Mini App that handles up to 10,000 active users:

- **RAM:** 2 GB minimum (4 GB recommended)
- **CPU:** 1–2 vCPUs
- **Storage:** 40 GB SSD
- **Bandwidth:** 1 TB/month or more
- **OS:** AlmaLinux 10 (or RHEL 9 / Rocky Linux 9 / CentOS Stream 9)

■■ **WARNING: IMPORTANT:** When buying VPS, choose **AlmaLinux 10** as the operating system. If AlmaLinux 10 is not available, choose **AlmaLinux 9**, **Rocky Linux 9**, or **RHEL 9**. All commands in this guide work on these systems. If you choose Ubuntu/Debian, many commands will be different (use **apt** instead of **dnf**).

✓ Step-by-Step: Buying a VPS (Hostinger Example)

Step 1: Go to hostinger.com/vps-hosting and click 'Buy Now' on the KVM 2 plan.

Step 2: Choose your billing period (12 or 24 months gives best discount).

Step 3: Create your account with your email.

Step 4: Pay with your card or PayPal.

Step 5: In the setup wizard, select:

- **OS:** AlmaLinux 10 (or AlmaLinux 9 if 10 not available)
- **Server Location:** Closest to your target users
- **Root password:** Create a STRONG password (save it safely!)

Step 6: Wait 2–5 minutes for your VPS to be provisioned. You'll get an email with your IP address.

■ **TIP:** Save your VPS details somewhere safe: **IP Address**, **Root Username** (usually *root*), and **Root Password**. You'll need these in the next section.

3. Initial Server Setup (AlmaLinux 10)

Now you'll connect to your VPS for the first time and prepare it for the app.

✓ Step 1: Connect to Your VPS via SSH

From Windows: Download **PuTTY** from putty.org. Open it, enter your VPS IP, click Open. Login as **root** with your password.

From Mac/Linux: Open Terminal and run:

```
ssh root@YOUR_VPS_IP_ADDRESS
```

From Hostinger panel: Click 'Browser Terminal' button — opens SSH right in your browser.

■ **NOTE:** First time you connect, you may see a 'fingerprint' warning. Type **yes** and press Enter. Then enter your root password (cursor won't show typing — that's normal).

✓ Step 2: Update System Packages

Once logged in, update everything:

```
dnf update -y
dnf install -y epel-release
dnf install -y curl wget git unzip nano firewalld policycoreutils-python-utils
```

This may take 2-3 minutes. When it finishes, run:

```
systemctl enable --now firewalld
```

✓ Step 3: Configure Firewall

Open the necessary ports:

```
firewall-cmd --permanent --add-service=http
firewall-cmd --permanent --add-service=https
firewall-cmd --permanent --add-service=ssh
firewall-cmd --reload
```

✓ Step 4: Set Hostname (Optional)

Give your server a nice name (replace *myserver* with whatever you want):

```
hostnamectl set-hostname myserver
```

✓ Step 5: Set Timezone

Set your timezone (example: Asia/Karachi). For other timezones, use `timedatectl list-timezones`.

```
timedatectl set-timezone Asia/Karachi
timedatectl
```

■ **TIP:** ■ Your server is now ready for software installation. Keep this SSH session open — we'll continue using it in the next section.

4. Installing Node.js, MySQL, Nginx

Now install the three main software packages your app needs.

✓ Step 1: Install Node.js (v20 LTS)

Node.js runs the backend (server.js, admin_server.js, bot.js):

```
curl -fsSL https://rpm.nodesource.com/setup_20.x | bash -
dnf install -y nodejs

# Verify installation
node --version
npm --version
```

You should see something like **v20.x.x** for node and **10.x.x** for npm.

✓ Step 2: Install MySQL 8

MySQL stores users, quizzes, withdrawals, etc.:

```
# Add MySQL repository
dnf install -y https://dev.mysql.com/get/mysql84-community-release-el9-1.noarch.rpm

# Disable default MySQL module (if present) – important on AlmaLinux 9/10
dnf module disable -y mysql 2>/dev/null || true

# Install MySQL Server
dnf install -y mysql-community-server

# Start and enable MySQL
systemctl enable --now mysqld

# Check status
systemctl status mysqld --no-pager
```

■ **NOTE:** If you see 'AlmaLinux 10' related errors, try the EL9 repo above (it works on AlmaLinux 10). If still failing, install *mysql-server* from base repo: *dnf install -y mysql-server*.

✓ Step 3: Secure MySQL

Get the temporary root password:

```
grep 'temporary password' /var/log/mysqld.log
```

You'll see a line ending with : **SOMERANDOMPASSWORD**. Copy that password.

Now run the security setup:

```
mysql_secure_installation
```

It will ask:

1. **Enter password:** Paste the temporary password
2. **New password:** Create a STRONG password (save it!)
3. **Re-enter:** Same password
4. **Remove anonymous users?** Press **Y**

5. **Disallow root login remotely?** Press Y
6. **Remove test database?** Press Y
7. **Reload privilege tables?** Press Y

■■ **WARNING: SAVE YOUR MYSQL PASSWORD!** You'll need it later when configuring the app. If you lose it, you'll need to reset MySQL completely.

✓ Step 4: Install Nginx

Nginx is the web server that handles HTTPS and routes traffic to your app:

```
dnf install -y nginx
systemctl enable --now nginx
```

```
# Test it's running
curl http://localhost
```

You should see the default Nginx welcome HTML output.

✓ Step 5: Install PM2 (Process Manager)

PM2 keeps your Node.js apps running 24/7 and auto-restarts them on crashes:

```
npm install -g pm2
```

```
# Verify
pm2 --version
```

■ **TIP:** ■ All required software is installed. Now you have: Node.js, MySQL, Nginx, and PM2 ready.

5. Connecting Your Domain to VPS

Now connect your domain (like yourapp.com) to your VPS IP so people can access your app via the domain name.

✓ Step 1: Buy a Domain (If You Don't Have One)

Recommended registrars:

- **Namecheap:** Best prices, great control panel
- **Hostinger:** If you're already there for VPS
- **GoDaddy:** Most popular but slightly more expensive
- **Cloudflare Registrar:** Cheapest renewals (no markup)

Cost: \$1–15/year depending on TLD (.com is more expensive than .xyz/.live).

✓ Step 2: Configure DNS — Point Domain to VPS IP

In your domain registrar's control panel, find **DNS Management** or **DNS Zone Editor**. Add these A records (use your real VPS IP):

Type	Name/Host	Value/Points To	TTL
A	@	YOUR_VPS_IP	Auto
A	www	YOUR_VPS_IP	Auto
A	admin	YOUR_VPS_IP	Auto

■ **NOTE:** Replace **YOUR_VPS_IP** with your actual VPS IP from Section 2 (e.g., 72.60.208.21). The **@** symbol means the root domain (yourapp.com itself). **www** and **admin** are subdomains (www.yourapp.com and admin.yourapp.com).

✓ Step 3: Wait for DNS Propagation

DNS changes take 5 minutes to 48 hours to propagate worldwide (usually 15-30 minutes). Check progress with:

```
# On your VPS:
dig yourdomain.com
dig admin.yourdomain.com

# Or use online tool:
# Visit: https://dnschecker.org/
# Enter your domain to see global DNS status
```

When ready, the result will show your VPS IP. Then test:

```
# Should return Nginx default page
curl http://yourdomain.com
```

■ **TIP:** ■ Your domain is now connected to your VPS. Next we'll add HTTPS (SSL) for security.

6. Setting Up Free SSL with Certbot

SSL (HTTPS) is required for Telegram Mini Apps. We'll use **Let's Encrypt** via Certbot — completely free and auto-renewing.

✓ Step 1: Install Certbot

```
dnf install -y certbot python3-certbot-nginx
```

✓ Step 2: Update Default Nginx Config First

Before getting SSL, Nginx must respond to your domain on HTTP. Edit the default config:

```
nano /etc/nginx/conf.d/yourapp.conf
```

Paste this (replace yourdomain.com with your actual domain):

```
server {
    listen 80;
    server_name yourdomain.com www.yourdomain.com admin.yourdomain.com;

    root /var/www/html;
    index index.html;

    location / {
        try_files $uri $uri/ =404;
    }
}
```

Save with **Ctrl+O**, Enter, then exit with **Ctrl+X**.

```
# Test config
nginx -t

# Reload Nginx
systemctl reload nginx
```

✓ Step 3: Get SSL Certificate

Run Certbot for both your main domain and admin subdomain:

```
certbot --nginx -d yourdomain.com -d www.yourdomain.com -d admin.yourdomain.com
```

It will ask:

1. **Email:** Your email (for renewal notices)
2. **Terms of Service:** Type **Y**
3. **Share email with EFF:** **N** (your choice)
4. **Redirect HTTP to HTTPS:** Type **2** (yes, redirect — recommended)

■ **TIP:** If successful, you'll see '**Successfully received certificate**'. Now visit <https://yourdomain.com> — should show padlock icon ■

✓ Step 4: Auto-Renewal Setup

Certbot auto-installs a renewal timer. Verify it's working:

```
systemctl status certbot-renew.timer
```

```
# Test renewal (dry run)
certbot renew --dry-run
```

■ **NOTE:** Let's Encrypt certificates expire every 90 days, but Certbot auto-renews them every 60 days. You don't need to do anything else — it's fully automatic.

7. Setting Up Database

Now create the database, import the schema, and configure access.

✓ Step 1: Login to MySQL

```
mysql -u root -p
# Enter the MySQL root password you set in Section 4
```

✓ Step 2: Create Database & User

Inside the MySQL prompt, run these commands (replace *yourapp_db* with your preferred name):

```
CREATE DATABASE yourapp_db CHARACTER SET utf8mb4 COLLATE utf8mb4_0900_ai_ci;

-- Grant root full access (we use root in this setup)
-- If you want a separate user, uncomment below:
-- CREATE USER 'appuser'@'localhost' IDENTIFIED BY 'StrongPassword123';
-- GRANT ALL PRIVILEGES ON yourapp_db.* TO 'appuser'@'localhost';

FLUSH PRIVILEGES;
EXIT;
```

✓ Step 3: Upload Project Files to VPS

From your computer, upload the project zip to your VPS using one of these methods:

Method A — File Manager (easiest): Hostinger has a built-in file manager. Navigate to `/home/`, upload `TelegramMiniApp.zip` there.

Method B — SCP Command (Mac/Linux):

```
scp TelegramMiniApp.zip root@YOUR_VPS_IP:/home/
```

Method C — WinSCP (Windows): Download winscp.net, connect with VPS IP and root password, drag-drop the zip into `/home/` folder.

✓ Step 4: Extract & Setup Project

Back in SSH terminal:

```
cd /home
unzip TelegramMiniApp.zip
mv TelegramMiniApp yourapp
cd yourapp

# Install dependencies
npm install

# Verify
ls -la
```

✓ Step 5: Import Database Schema

```
mysql -u root -p yourapp_db < database.sql
```

```
# Verify tables imported
```

```
mysql -u root -p yourapp_db -e "SHOW TABLES;"
```

```
# Should list: admin, ads_config, notifications, quiz_results, quizzes, users,
withdrawals
```

■ **TIP:** Database is set up with demo data and 5+ sample quizzes. You can add/edit quizzes from the admin panel later.

✓ Step 6: Update Default Admin Password

The default admin password is **CHANGE_THIS_PASSWORD**. Update it now:

```
mysql -u root -p yourapp_db -e "UPDATE admin SET password='YourNewStrongPassword123'
WHERE username='admin';"
```

■■ **WARNING:** Use a strong password — minimum 12 characters, mix of letters, numbers, and symbols. Save it in a password manager. You'll use this to log into the admin panel.

8. Configuring Project Code

Now replace all the placeholder values in the code with your actual credentials.

✓ Files to Configure

File	What to Change
server.js	MySQL password, database name
admin_server.js	MySQL password, database name
bot.js	Bot token, MySQL password, database name
telegram/*.html	Domain (yourdomain.com)
admin/*.html	Admin domain (admin.yourdomain.com)
user/index.html	Domain, branding, contact info

✓ Method: Find & Replace via sed (Fastest)

Use **sed** to replace all placeholders at once across the entire project:

```
cd /home/yourapp

# Replace MySQL password (replace YourMySQLPass123 with your actual MySQL password)
sed -i 's|YOUR_MYSQL_PASSWORD_HERE|YourMySQLPass123|g' server.js admin_server.js bot.js

# Replace database name (replace yourapp_db with the name you created in Section 7)
sed -i 's|your_database_name|yourapp_db|g' server.js admin_server.js bot.js database.sql

# Replace bot token (we'll get this in Section 9 – come back to this command)
sed -i "s|YOUR_BOT_TOKEN_HERE|YOUR_ACTUAL_BOT_TOKEN|g" bot.js

# Replace domains
sed -i 's|yourdomain.com|yourapp.com|g' admin/*.html user/*.html telegram/*.html
sed -i 's|admin.yourapp.com|admin.yourapp.com|g' admin/*.html telegram/*.html
```

■■ **WARNING:** Do **NOT** run the bot token replacement yet — we'll create the bot in the next section and get a real token. Come back and run that command after you have the bot token.

✓ Manual Edit Method (If sed Fails)

Open each file with nano and find/replace manually:

```
nano server.js
# In nano, press Ctrl+W to search, type YOUR_MYSQL_PASSWORD_HERE, press Enter
# Replace with your password manually
# Save: Ctrl+O, Enter, Exit: Ctrl+X

# Repeat for admin_server.js and bot.js
```

✓ Update Branding (User-Facing)

Replace 'YourAppName' with your actual app name across all files:

```
sed -i 's|YourAppName|MyAwesomeQuizApp|g' user/index.html admin/*.html telegram/*.html bot.js
```

Replace your contact info (your Telegram username):

```
sed -i 's|@your_telegram_username|@your_real_telegram|g' telegram/dashboard.html
```

■ **TIP:** ■ Code is now configured. Next: create your Telegram Bot to get the BOT_TOKEN.

9. Creating Your Telegram Bot

Every Telegram Mini App needs a Bot. Bots are created via **@BotFather** — Telegram's official bot creation tool.

✓ Step 1: Talk to BotFather

1. Open Telegram, search for **@BotFather**
2. Click 'Start'
3. Send: **/newbot**
4. **Bot name:** Type the display name (e.g., 'My Quiz App')
5. **Bot username:** Type a unique username ending with 'bot' (e.g., 'MyQuizApp_bot')

■ **NOTE:** Bot username MUST end with **bot** or **Bot** and must be unique across Telegram. If your name is taken, try variations like MyQuizApp2025_bot.

BotFather replies with a token like:

8266167780:AAGHVCZj04fNTX7iFsyoXNopo9a-w0LiuEw

■■ **WARNING: SAVE THIS TOKEN — DO NOT SHARE IT!** Anyone with this token can control your bot. Treat it like a password. Save it in a password manager.

✓ Step 2: Configure Bot Settings (Important!)

Still in BotFather chat, run these commands one by one:

Command	Purpose
/setdescription	Description shown when users find your bot
/setabouttext	Short bio (max 120 chars)
/setuserpic	Upload a logo for your bot
/setcommands	Commands list (e.g., 'start - Open the app')
/setmenubutton	Add Menu Button to launch your mini app

✓ Step 3: Save Token in Code

Back in your VPS SSH session:

```
cd /home/yourapp
sed -i "s|YOUR_BOT_TOKEN_HERE|YOUR_ACTUAL_BOT_TOKEN|g" bot.js

# Verify
grep "token =" bot.js
```

Replace *YOUR_ACTUAL_BOT_TOKEN* with the real token from BotFather (don't include quotes when running sed; the script adds them).

✓ Step 4: Configure Mini App Menu Button

In @BotFather, send **/setmenubutton**, choose your bot, then:

1. **Button text:** 'Open App' (or whatever you want)
2. **Button URL:** <https://yourdomain.com/telegram/dashboard.html>

■ **TIP:** Now when users open your bot, they'll see a Menu Button at the bottom that launches your Mini App!

10. Configuring Nginx (Reverse Proxy)

Nginx receives all web traffic and routes it to the right Node.js server. We'll set up two domains: **yourdomain.com** (user app) and **admin.yourdomain.com** (admin panel).

✓ Step 1: Edit Nginx Configuration

```
nano /etc/nginx/conf.d/yourapp.conf
```

Replace ALL contents with this configuration (replace *yourdomain.com* with your actual domain):

```
server {
    listen 80;
    server_name yourdomain.com www.yourdomain.com;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl http2;
    server_name yourdomain.com www.yourdomain.com;

    # SSL – managed by Certbot
    ssl_certificate /etc/letsencrypt/live/yourdomain.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/yourdomain.com/privkey.pem;
    include /etc/letsencrypt/options-ssl-nginx.conf;
    ssl_dhparam /etc/letsencrypt/ssl-dhparams.pem;

    # Disable caching for HTML files (prevents stale code on user devices)
    location ~* \.html$ {
        add_header Cache-Control "no-store, no-cache, must-revalidate, max-age=0"
always;
        add_header Pragma "no-cache" always;
        expires -1;
        root /home/yourapp;
        try_files $uri =404;
    }

    # Telegram mini app pages
    location /telegram/ {
        root /home/yourapp;
        try_files $uri $uri/ =404;
    }

    # API requests go to Node.js (port 3000)
    location /api/ {
        proxy_pass http://localhost:3000;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    # Public landing page
    location / {
```

```

        root /home/yourapp/user;
        try_files $uri $uri/ /index.html;
    }
}

server {
    listen 80;
    server_name admin.yourdomain.com;
    return 301 https://$host$request_uri;
}

server {
    listen 443 ssl http2;
    server_name admin.yourdomain.com;

    ssl_certificate /etc/letsencrypt/live/yourdomain.com/fullchain.pem;
    ssl_certificate_key /etc/letsencrypt/live/yourdomain.com/privkey.pem;
    include /etc/letsencrypt/options-ssl-nginx.conf;
    ssl_dhparam /etc/letsencrypt/ssl-dhparams.pem;

    # Admin API requests go to admin server (port 4000)
    location /api/ {
        proxy_pass http://localhost:4000;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }

    # Admin panel pages
    location / {
        root /home/yourapp/admin;
        try_files $uri $uri/ /index.html;
    }
}

```

✓ Step 2: Test & Reload Nginx

```

# Test config syntax
nginx -t

# If "syntax is ok" and "test is successful", reload:
systemctl reload nginx

```

✓ Step 3: Configure SELinux (CRITICAL on AlmaLinux!)

On AlmaLinux/RHEL, SELinux blocks Nginx from connecting to Node.js by default. Allow it:

```

# Allow Nginx to connect to local services (Node.js)
setsebool -P httpd_can_network_connect 1

# Allow Nginx to read project files
chcon -R -t httpd_sys_content_t /home/yourapp/

# Verify
getsebool httpd_can_network_connect

```

■■ **WARNING: VERY IMPORTANT:** Without these SELinux commands, you'll get **502 Bad Gateway** errors when Nginx tries to reach your Node.js apps. This is the #1 issue on AlmaLinux deployments.

■ **TIP:** ■ Nginx is configured. Next we'll start the Node.js apps with PM2.

11. Deploying with PM2

PM2 keeps your Node.js apps running 24/7. We'll start three processes: user server, admin server, and bot.

✓ Step 1: Start All Processes

```
cd /home/yourapp

# Start user API server (port 3000)
pm2 start server.js --name yourapp-user

# Start admin API server (port 4000)
pm2 start admin_server.js --name yourapp-admin

# Start Telegram bot
pm2 start bot.js --name yourapp-bot

# Save current process list
pm2 save

# Setup auto-start on server reboot
pm2 startup systemd
# Run the command it outputs (it gives you a command to copy-paste)
```

✓ Step 2: Verify All Running

```
pm2 list
```

You should see all 3 processes with status **online**:

#	Name	Status	Description
0	yourapp-user	online	User API on port 3000
1	yourapp-admin	online	Admin API on port 4000
2	yourapp-bot	online	Telegram bot polling

✓ Step 3: Check Logs (If Issues)

```
# View all logs
pm2 logs

# View specific process logs
pm2 logs yourapp-user
pm2 logs yourapp-admin
pm2 logs yourapp-bot

# Last 50 lines only
pm2 logs --lines 50

# Stop tailing: Ctrl+C
```

✓ Step 4: Useful PM2 Commands

Command	Purpose
pm2 list	Show all processes
pm2 restart all	Restart all apps
pm2 restart yourapp-user	Restart specific app
pm2 stop all	Stop all apps
pm2 delete all	Remove all apps from PM2
pm2 monit	Real-time monitoring dashboard
pm2 logs --err	Show error logs only

■ **TIP:** ■ Your app is now running 24/7! It will automatically restart if anything crashes, and start back up if the server reboots.

12. Linking Bot to Mini App

Final connection: tell BotFather your domain so users can launch the mini app.

✓ Step 1: Configure Web App URL

In @BotFather, send: **/mybots**

1. Select your bot
2. Click **Bot Settings** → **Menu Button** → **Configure menu button**
3. Send button text: **Open App**
4. Send URL: **<https://yourdomain.com/telegram/dashboard.html>**

✓ Step 2: Set Domain (For Mini App Validation)

In @BotFather:

1. **/mybots** → Select your bot
2. **Bot Settings** → **Configure Mini App**
3. Click **Enable Mini App**
4. Send your URL: **<https://yourdomain.com/telegram/dashboard.html>**

✓ Step 3: Test Your Bot

Open Telegram, search for your bot's username, click Start. You should see:

- Welcome message from bot
- Menu Button at bottom (Open App)
- Click Menu Button → Mini App opens with your dashboard ■

■ **TIP:** ■ **YOUR APP IS LIVE!** Users can now find your bot, click 'Open App', and play quizzes to earn rewards.

13. Going Live — Final Tests

Before announcing your app, run through this checklist to make sure everything works:

Test	How	Expected
Domain loads	Visit https://yourdomain.com	Landing page shows
SSL works	Browser shows padlock icon	■ Green padlock
Admin login	Visit https://admin.yourdomain.com	Login page shows
Admin works	Login with admin credentials	Dashboard loads
Bot replies	Send /start to your bot	Welcome message
Mini app opens	Click Menu Button in bot	Dashboard shows your name
Quiz works	Click Quiz, answer questions	Coins added to balance
Withdraw works	Submit withdraw request	Appears in admin panel

✓ Add Initial Quizzes

1. Login to **admin.yourdomain.com**
2. Go to **Quizzes** section
3. Click **Bulk Upload Quizzes**
4. Paste a JSON array of quizzes
5. Click **Upload All Quizzes**

✓ Configure Ads (Optional)

If you want to monetize with ads:

1. Sign up at **monetag.com** or **adsterra.com**
2. Get your ad codes (script tags)
3. In admin panel → **Ads Config**
4. Paste your ad code in each placement
5. Save

■ **TIP:** All tests passing? Time to launch! Share your bot link in Telegram channels, social media, etc. and watch users come in.

14. Troubleshooting Common Issues

■ '502 Bad Gateway' Error

Cause: Nginx can't reach Node.js apps. Either Node.js crashed or SELinux blocking.

```
# Check if Node.js apps are running
pm2 list

# If 'errored' or 'stopped', check logs:
pm2 logs --err --lines 30

# Common fix #1 – SELinux blocking (AlmaLinux/RHEL):
setsebool -P httpd_can_network_connect 1

# Common fix #2 – Module missing:
cd /home/yourapp
npm install
pm2 restart all
```

■ Bot Not Responding to /start

```
# Check bot logs
pm2 logs yourapp-bot --lines 50

# Common issue: wrong token or token has spaces
grep "token" /home/yourapp/bot.js

# Restart bot
pm2 restart yourapp-bot
```

■ MySQL 'Access denied' Error

```
# Test MySQL connection manually
mysql -u root -p

# If password works in MySQL but not in app:
# Check there are no special chars escaping in code
nano /home/yourapp/server.js
# Look for password: "... " line – should match exactly
```

■ Mini App Shows 'Open via Telegram'

Cause: Telegram WebApp data isn't loading. Usually a browser cache issue.

Fix:

1. Telegram → Settings → Data and Storage → Clear Cache
2. Block bot, then unblock
3. Send /start again
4. Open Mini App

■ SSL Certificate Errors

```
# Renew certificate manually
certbot renew --force-renewal

# If domain validation fails, ensure DNS is correct:
dig yourdomain.com

# Test config
nginx -t
systemctl reload nginx
```

■ Database Tables Missing

```
# Re-import database
cd /home/yourapp
mysql -u root -p yourapp_db < database.sql

# Verify
mysql -u root -p yourapp_db -e "SHOW TABLES;"
```

■ Slow Performance

```
# Check server resources
free -h          # RAM usage
df -h            # Disk usage
top              # CPU usage (Ctrl+C to exit)

# Check Nginx error log
tail -50 /var/log/nginx/error.log

# Check API speed
time curl https://yourdomain.com/api/quizzes/123456789
```

■ Firewall Blocking

```
# Check open ports
firewall-cmd --list-all

# Should include: http https ssh
# If missing:
firewall-cmd --permanent --add-service=http
firewall-cmd --permanent --add-service=https
firewall-cmd --reload
```

15. Maintenance & Updates

✓ Backing Up Your Database

Take regular backups (recommended weekly):

```
# Manual backup
mysqldump -u root -p yourapp_db > /root/backup_$(date +%Y%m%d).sql

# Compress it
gzip /root/backup_$(date +%Y%m%d).sql

# Auto backup with cron (runs every Sunday at 2 AM)
crontab -e

# Add this line:
0 2 * * 0 mysqldump -u root -pYourMySQLPass yourapp_db | gzip > /root/backup_$(date +%Y%m%d).sql.gz
```

✓ Updating Code

```
cd /home/yourapp

# 1. Backup current code
cp -r /home/yourapp /home/yourapp_backup

# 2. Upload new files via SCP/SFTP/FileManager

# 3. Install any new dependencies
npm install

# 4. Restart all
pm2 restart all
```

✓ Monitoring

```
# Real-time PM2 monitor
pm2 monit

# Check disk space
df -h

# Check who's logged into MySQL
mysql -u root -p -e "SHOW PROCESSLIST;"

# Check Nginx access log
tail -f /var/log/nginx/access.log
```

✓ Security Best Practices

Practice	How
Disable root SSH after setup	Create non-root user, edit /etc/ssh/sshd_config
Use SSH keys instead of passwords	ssh-keygen + ssh-copy-id
Keep system updated	dnf update -y (run weekly)
Strong MySQL password	Min 16 characters, mixed
Different admin password	Change CHANGE_THIS_PASSWORD immediately
Backup regularly	Daily/weekly database dumps
Monitor logs	pm2 logs, tail nginx logs
Use Cloudflare	Free DDoS protection and CDN

■■■ About the Author

Built with passion by

Mr Naveed Shah

■ YouTube Channel

[@shahgideas](#)

youtube.com/@shahgideas

Channel Name: SHAHG

■ Why This Project Exists

This complete Telegram Mini App took **months of development, debugging, and refinement**. From handling identity bugs to optimizing performance, every feature has been battle-tested in production.

I'm releasing it **completely free** to the community so anyone can build their own earning app without starting from scratch. If this guide and project helped you:

■ Support the Author

- ✓ Subscribe to YouTube: [@shahgideas](#)
- ✓ Like videos and leave a comment
- ✓ Share this guide with other developers
- ✓ Follow for more dev tutorials & projects

Your support motivates me to create more open-source projects and share knowledge with the developer community. Thank you for being part of this journey!



■ Visit: youtube.com/@shahgideas

■ Congratulations!

You now have a fully functional Telegram Mini App running on your own VPS, secured with SSL, and managed with PM2.

■ Resources

Telegram Bot API: <https://core.telegram.org/bots>

Mini Apps Docs: <https://core.telegram.org/bots/webapps>

Node.js Docs: <https://nodejs.org/docs>

MySQL Docs: <https://dev.mysql.com/doc>

Nginx Docs: <https://nginx.org/en/docs>

PM2 Docs: <https://pm2.keymetrics.io/docs>

Let's Encrypt: <https://letsencrypt.org/docs>

This guide is open-source. Feel free to share, modify, and use it however you wish. If you find issues or have suggestions, contributions are welcome!